

JSensor: Um simulador paralelo para redes de sensores em larga escala

Matheus L. Silva¹, Dannel H. Ribeiro¹, Joubert C. Lima¹
Andre L. L. Aquino², Ricardo A. R. Oliveira¹

¹Departamento de computação – Universidade Federal de Ouro Preto (UFOP)
Ouro Preto – MG – Brasil

²Instituto de Computação – Universidade Federal de Alagoas (UFAL)
Maceió – AL – Brasil.

matnetmg@yahoo.com.br, {danielhugo, joubertlima}@gmail.com

{alla.lins, rrabelo}@gmail.com

Abstract. *This paper presents a large scale sensor network simulator called JSensor which executes parallel simulations for shared memory or multi-core architectures. JSensor allows asynchronous large scale sensor networks simulations. The application code level of concurrence on the tested shared memory is close to 90 – 95%, with super linear speed-up as the number of computer cores increases, and near-linear runtime function of the number of nodes up to one million and six hundred thousand nodes flooding data in our tested scenario. JSensor consumes about 4 GB of RAM to simulate 1.6 million flooding sensors.*

Resumo. *Este artigo apresenta o JSensor, uma ferramenta para simulação de rede de sensores sem fio em larga escala que realiza simulações de forma paralela, em arquiteturas de memória compartilhada ou multicore. JSensor se mostrou eficiente, com simulações assíncronas com tempo de execução próximo ao linear e um speedup super linear para um nível de concorrência de aproximadamente 90 – 95%. Estes resultados foram obtidos considerando um cenário com um milhão e seiscentos mil nós enviando dados do tipo inundação. O JSensor consome cerca de 4 GB de RAM para simular 1,6 milhões de sensores.*

1. Descrição e motivação do problema tratado pelo JSensor

Redes de sensores sem fio (RSSFs) com milhares de nós sempre esteve presente nas discussões a respeito da viabilidade da utilização de tais redes em ambientes reais. Com o advento da combinação desses sensores inteligentes com aplicações de monitoramento ambiental, redes veiculares, cidades inteligentes etc, tal discussão tornou-se mais presente. Por essa razão, a simulação de aplicações de RSSFs em larga escala tem se tornado uma desafiadora área [Khan et al. 2011]. Tais aplicações estão demandando novas ferramentas de simulação que permitam e suportem o desenvolvimento de modelos de comunicação e distribuição de nós, roteamento e processamento de dados em redes com milhares de nós.

Recentemente, algumas aplicações reais para o monitoramento ambiental com RSSFs propõem o uso de centenas de nós. Segundo Oliveira et

al. [Oliveira and Rodrigues 2011], uma RSSFs deve acomodar milhares de nós, logo torna-se necessário se mostrar que as soluções teóricas disponíveis são adequadas para essas redes. Para isso, ferramentas que possam simular tais ambientes tornam-se necessárias. A título de exemplo, se estivermos interessados em efetuar um micro monitoramento ambiental na floresta amazônica, seria necessário a utilização de milhares de nós para cobrir uma grande área. Outro exemplo, se quisermos realizar um micro monitoramento da qualidade do ar no estado de São Paulo, mais uma vez, seria necessário a utilização de milhares de nós. Vale destacar que tanto a floresta amazônica quanto o estado de São Paulo possuem estações de sensoriamento que efetuam um monitoramento macro.

A simulação de um grande número de nós pode causar um grande impacto nas métricas de desempenho das ferramentas de simulação caso elas não sejam projetadas para tal. O usuário ou projetista almeja construir aplicações RSSFs cada vez maiores e mais rápidas. Esta demanda traz desafios ao simulador como, por exemplo, como garantir robustez e segurança no mecanismo de comunicação entre os sensores, qual a localidade dos sensores, qual o nível de concorrência dos processos do simulador, como balancear a carga para garantir maior equidade na alocação de processos do Sistema Operacional, como garantir dependências causais entre sensores sendo executados por diferentes processos e como oferecer tolerância a falhas de comunicação ou processamento dos sensores. Construir simuladores que combinam tais funcionalidades e ainda permitam a modelagem de sensores cada vez mais reais, isto é, com elevado número de instruções, complexos e custosos computacionalmente, é um enorme desafio.

Neste artigo apresentamos um simulador em paralelo de RSSFs em larga escala, chamada JSensor (<http://www.joubertlima.com.br/JSensor/>). Esse simulador é uma extensão do Sinalgo (<http://disco.ethz.ch/projects/sinalgo>) que é um arcabouço para teste e validação de algoritmos distribuídos e que foi descontinuado em 2008. O JSensor foi executado em máquinas com múltiplos núcleos de processamento e mostrou-se eficiente, com um tempo de execução próximo ao linear e com um *speedup* super linear, refletindo um nível de concorrência no simulador de aproximadamente 90 – 95%. O consumo de memória não ultrapassou os 4 GB de RAM. Os resultados foram obtidos a partir de experimentos com até 1.6 milhão de sensores enviando dados do tipo inundação.

Para mostrar a eficiência e as limitações do JSensor o comparamos com o OMNeT++ (<http://www.omnetpp.org>) que é um simulador orientado a objeto para simulação discreta, escrito em linguagem C++ e com interface gráfica 2D. De forma geral, tanto o simulador quanto as interfaces são altamente portáteis, podendo ser executados em qualquer Sistema Operacional. Possui uma extensão chamada *Horizon* que permite executar simulações em arquiteturas multiprocessadas e as extensões *Castalia* e *MiXiM* que permitem simular RSSFs. No entanto, tanto o *Castalia* como o *MiXiM* executam sobre o núcleo OMNeT++ e não sobre o *Horizon*, com isto propriedades específicas de RSSFs não podem ser executadas em paralelo. Portanto, diferente do JSensor, o OMNeT++ não permite o paralelismo direto de cenários baseados em RSSFs, implementados no *Castalia*. Assim, caso seja necessário a simulação de cenários com requisitos específicos de RSSFs o projetista deverá reimplementá-los e revalidá-los diretamente no *Horizon*.

Além do OMNet++, identificamos alguns simuladores com características específicas, como a escalabilidade e capazes de executar em ambientes paralelos [Lee et al. 2009, Lewis et al. 2002]. Ao contrário do JSensor, tais simuladores não apresentam características específicas de RSSFs ou não simulam um grande número de nós.

2. Arquitetura do JSensor e descrição das principais funcionalidades

O JSensor é totalmente escrito em Java, o que o torna portátil para qualquer plataforma. Ele permite a execução de simulações assíncrona que são puramente baseada em eventos. O núcleo possui um conjunto de eventos de envio/recepção de mensagens e de temporizadores, os quais são ordenados pelo tempo em que devem ocorrer. Repetidamente, o simulador retira o evento mais recente da lista e o executa.

Toda a interface gráfica do JSensor é baseada na interface do Sinalgo. Dessa forma, as simulações podem ser visualizadas e manipuladas graficamente via GUI ou podem ser executadas em blocos via linha de comando.

Para implementar cenários de RSSFs em paralelo, o JSensor provê um conjunto de classes que, em conjunto com as classes já existentes no Sinalgo, podem ser utilizadas pelos projetistas para implementar suas simulações. A Figura 1 ilustra o conjunto de classes base do JSensor, bem como suas relações. O JSensor possui uma fachada representada pela classe *Runtime* que é responsável pelo gerenciamento das múltiplas *threads* que simulam os sensores em paralelo.

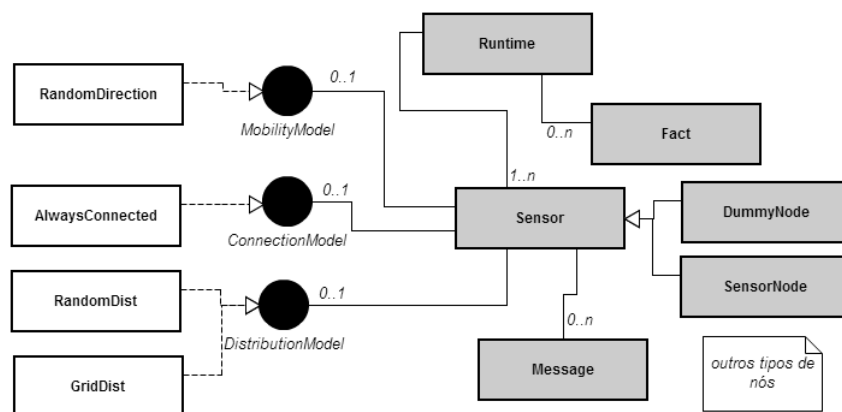


Figura 1. Principais classes do JSensor.

Além da classe *Runtime*, que será detalhada mais adiante, temos a classe abstrata *Node* que representa os nós sensores da rede. Todo novo tipo de nó criado deve estender a classe *Node*. As seguintes classes concretas derivadas de *Node* estão disponíveis no JSensor: (i) *DummyNode* que é o nó básico que não executa nada específico; (ii) *SensorNode* que é responsável por detectar eventos e coletar dados; (iii) *RobotNode* que é responsável por coletar os dados sensorizados pelo nó *SensorNode*; (iv) *RouteNode* que é responsável por rotear mensagens em tráfego; e (v) *SinkNode* que é o nó destino para todos os dados coletados. De uma forma geral, cada nó possui uma série de métodos que lhe conferem cálculo de vizinhança, estado inicial, comunicação e ações específica do projetista da rede.

A classe abstrata `Message` é usada para implementar as mensagens enviadas pelo nó durante a simulação. A classe `Message` define o tipo de mensagem e a forma como elas são criadas. O único método da classe `Message` é o `clone()` que retorna uma cópia idêntica da mensagem anterior. Note que o usuário poderá descrever qualquer mensagem no `JSensor`, apenas herdando a classe `Message`. Como padrão o `JSensor` permite o envio de dois tipos de mensagens: *flooding* ou *unicast*.

A classe abstrata `Fact` representa os eventos que ocorrem no ambiente monitorado, chuva, fogo, e tantos outros fenômenos podem ser modelados como fatos no `JSensor`. Se um fato expira ele é removido automaticamente da simulação. Cada fato é associado a um ou mais `AbstractFactEvent` que é uma classe abstrata responsável por atualizar os valores dos fatos.

Além das classes descritas acima, o `JSensor` permite a especificação de modelos para distribuição e conectividade dos nós. O modelo de *distribuição do nó* garante posição geográfica inicial em uma área monitorada. O modelo de distribuição requer que o usuário implemente apenas o método `getNextPosition()` que retorna uma posição inicial para um dado nó. Como padrão o `JSensor` possui implementado os modelos de distribuição: (i) `RandomDistribution` que distribui aleatoriamente os nós na área monitorada; e (ii) `GridDistribution` que distribui os nós numa grade.

O modelo de *conectividade dos nós* permite a manipulação da conectividade entre os nós. Para isso, os usuários possuem os seguintes métodos: `updateConnection(Node n, ...)`, para um dado nó n , este método atualiza sua lista de vizinhos e retorna `true` se ocorreu alguma mudança nos vizinhos e `false` caso contrário; `isNear(Node  $n_1$ , Node  $n_2$ )` verifica se dois nós estão próximos um do outro, considerando o raio de comunicação; e `isConnected(Node from, Node to)` define quando dois nós estão conectados. Apenas este método deve ser implementado pelo usuário. Como padrão o `JSensor` disponibiliza o modelo de conectividade `AlwaysConnected` que permite a conexão de um nó com todos os que estão no seu raio de comunicação.

A Figura 2 ilustra a estratégia `JSensor` para paralelizar as simulações. O núcleo do `JSensor` possui uma fachada representada pela classe `Runtime` que é responsável pelo gerenciamento de múltiplas *threads* (*Worker Threads*) simulando sensores em paralelo. O número de *threads* criadas é sempre igual ao número de processadores na máquina. Cada *Worker Thread* gerencia um conjunto de sensores proporcional ao número de processadores na máquina. Não há qualquer particionamento baseado na posição do sensor. Com isto, evitamos desbalanceamento de carga em cenários onde aglomerados de sensores se formam na grade de simulação.

Cabe a classe `Runtime` manter ordenado os tempos de todos os eventos produzidos pelos sensores. O `JSensor` cria uma barreira de sincronização entre suas *threads* somente nos tempos especificados pelos sensores ao criarem seus eventos futuros. Desta forma, garantimos a simulação paralela de todos os sensores que possuem eventos no tempo corrente do simulador. O `JSensor` compartilha uma grade de células entre as *threads* e, conseqüentemente, entre os sensores. Cada célula possui as coordenadas `lat`, `long`, `alt` e quais sensores estão nesta posição do globo. Desta forma, o `JSensor` representa um espaço compartilhado e seguro onde os sensores residem, montado sob

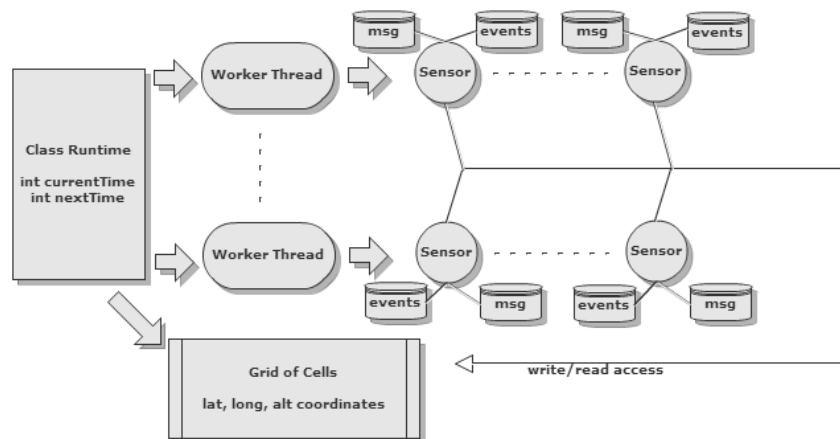


Figura 2. Execução do JSensor.

demanda, pois uma célula só existe se há pelo menos um sensor residindo nela.

O Algoritmo 1 ilustra a execução da classe `Runtime`. Como pode ser observado, só é necessário que o usuário especifique o conjunto de sensores $S_{k,n/k}$, onde n/k são os números de sensores que executam no processador k , e o tempo de simulação $time$. Ademais, os possíveis estados que um nó pode assumir em um determinado evento é representado por σ que possui valores pré-definidos: σ_1 processando uma mensagem; σ_2 atualizando lista de vizinhos; e σ_3 atualizando lista de eventos.

Algorithm 1 Execução da classe `Runtime`

Entrada: $time, S$

```

1: enquanto  $t \leq time$  faça
2:    $event \leftarrow$  evento da lista de eventos a ser executado no instante  $t$ 
3:   para  $i \leftarrow 0 \dots |\sigma|$  faça
4:     para  $j \leftarrow 0 \dots k$  faça
5:       Executa o estado  $\sigma_i$  de todos os nós  $S_{j,*}$  que fazem parte do evento  $event$ 
6:     fim para
7:   Sincroniza todos  $S$  que executaram  $\sigma_i$ 
8:   fim para
9: fim enquanto
  
```

De forma detalhada temos os seguinte passos no Algoritmo 1: Linhas 1 - 9 temos a execução da simulação controlada pelo tempo especificado pelo usuário; Linha 2 consome o evento a ser executado no instante t . Tal evento será representado e executado por um conjunto específico de sensores; Linhas 3 - 8 temos a verificação de todos os estados que cada sensor deve assumir durante a execução de cada evento. Para cada transição de estado todas as *threads* varrerem todos os nós para tentarem simula-los (linhas 4 - 6). Assim, haverá execuções concorrentes dos eventos de um tipo pré-definido e anteriores ao instante t apenas. De forma mais específica, implementamos um *pool de threads* pra evitar a criação e remoção de *threads* de forma recorrente. Com o *pool de threads* mantemos algumas *threads* ativas enquanto a simulação durar. A linha 7 é a barreira de sincronização das *threads*. A cada grupo iniciado de *threads* para cada estado existe a necessidade de aguardar que todas as demais *threads* terminem antes que haja a mudança de estado. Desta forma, o JSensor garante que os sensores podem se mover, processar, enviar/receber mensagens e criar novos eventos em paralelo, porém um evento por vez.

3. Avaliação do desempenho

Nesta seção avaliamos o JSensor vs. OMNeT++ considerando até 1.600.000 (um milhão e seiscentos mil) sensores. As mensagens enviadas são do tipo inundação. De forma aleatória cada sensor manda uma mensagem para o sorvedouro seguindo a estratégia de inundação, ou seja, cada sensor envia a mensagem para o seu conjunto de vizinhos que reenvia para os seus vizinhos até que a mensagem chegue ao sorvedouro. Sob o ponto de vista prático essa estratégia é pouco utilizada. Consideramos a inundação na avaliação de desempenho do JSensor, pois essa estratégia de propagação de dados gera um alto tráfego na rede, exigindo bastante dos recursos do simulador, onde 100% dos nós, se alcançáveis, participam da simulação.

Como o JSensor permite a concorrência entre sensores com eventos do mesmo tipo e num mesmo tempo, durante a simulação há apenas um tipo de evento e portanto os sensores enviam mensagens a seus vizinhos concorrentemente quando seus eventos ocorrem simultaneamente. Vale salientar que não estamos tratando colisão, ou seja, todas as mensagens enviadas são recebidas e propagadas. Após os sensores serem criados a aplicação inicia o envio de mensagens por intermédio de 10 mil nós.

Desta forma, em pouco tempo todos os sensores possuem mensagens para consumir e enviar. Nos cenários simulados cada sensor não possui mais do que $d = 8$ sensores como seus vizinhos, onde $d = \pi R N / L^2$, $R = 10$ é o raio de comunicação, $N = \{100k, 200k, 400k, 800k, 1.600k\}$ o número de sensores e L^2 é a área de sensoramento e varia de acordo com o número de nós [Aquino et al. 2013].

Os experimentos foram executados em uma máquina com as seguintes configurações: dois núcleos Intel Xeon E5405 quad-core com 2 GHz; 16 GB de RAM compartilhada; quatro drives SATAII com 7200 rpm; Sistema Operacional Linux Ubuntu 12.01 LTS 64-bit; e todos os algoritmos foram codificados com o java 64-bit (JRE 1.7.0_07). Cada simulação foi executada quinze vezes. Analisamos o JSensor quanto a seu tempo de execução, consumo de memória e aceleração.

O JSensor consome cerca de 60% da memória que OMNeT++ consome quando o número de sensores ultrapassa 1M (um milhão), porém para baixo número de sensores, ou seja, menos que 300k (trezentos mil) sensores, o JSensor necessita de um pouco mais de memória que o OMNeT++. A Figura 3 apresenta o comportamento descrito acima. Isso ocorre pois até certo ponto a complexidade introduzida pelo JSensor para garantir paralelismo o afeta no consumo de memória, porém este custo não aumenta muito a medida que o número de sensores se eleva. Em contrapartida, a complexidade dos sensores no OMNeT++ o afeta à medida que o número de sensores aumenta. Este detalhamento obrigatório no OMNeT++ afeta seu consumo de memória, mesmo quando muitas funcionalidades nativas do sensor não estão sendo avaliadas na simulação.

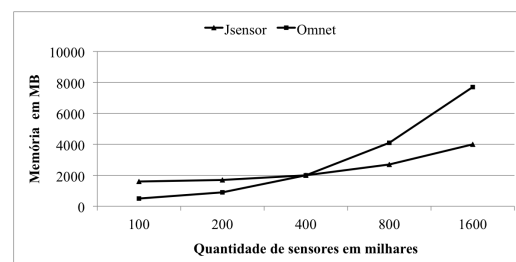


Figura 3. Utilização de memória.

A Figura 4 ilustra as medidas para o tempo de execução quando simulam 100-

1.600k sensores do tipo inundação. Na Figura 4(a) para poucas centenas de milhares de sensores, o JSensor se mostra eficiente. Porém, a partir de 800k sensores o JSensor é penalizado no seu tempo de execução em relação ao OMNeT++. Isso se deve ao tempo de criação dos sensores que ainda se apresenta bastante custoso no JSensor, conforme pode ser observado na Figura 4(b) que mostra os tempos de simulação e criação de sensores no JSensor para 100-1600k sensores. O custo de simular a execução de um sensor passa a ser muito menor que o custo de criá-lo. Note que se desconsiderarmos o tempo necessário para a criação dos sensores, o JSensor ficaria com aproximadamente a metade do tempo de execução do OMNeT++.

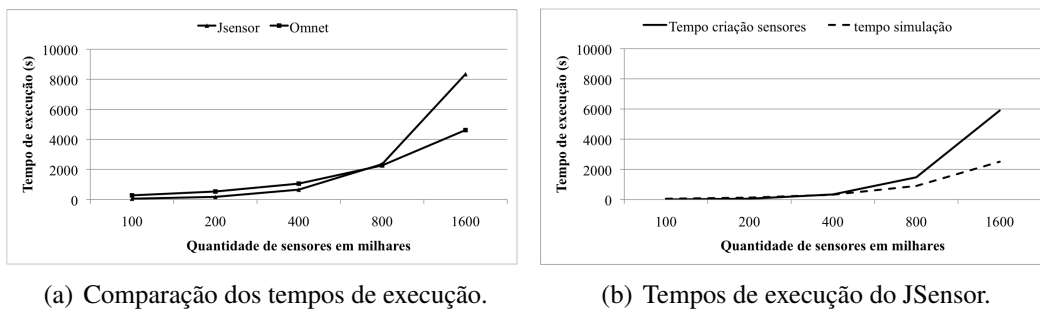


Figura 4. Avaliação dos tempos de execução.

Realizamos também testes para medir a aceleração do JSensor. Nos testes variamos de 1-8 o número de *threads* JSensor e com 100k e 1600k o número de sensores. Podemos aferir que o JSensor é escalável, possuindo aceleração superlinear, com nível de paralelismo na ordem de 90-95% para 100k sensores, segundo a lei de Amdahl. A Figura 5 mostra que a aceleração melhora à medida que o número de sensores aumenta. Este comportamento foi formalmente explicado pela lei de Gustafson [Gustafson 1988], que detectou que o nível de paralelismo varia conforme o tamanho da entrada, pois a porção sequencial sempre fica menos significativa à medida que o custo computacional total aumenta.

O comportamento superlinear pode ser explicado pela exaustiva execução de criação de sensores em paralelo. O nível de repetição de instruções é altíssimo e favorável a *cache-hit*. Tais instruções já podem ser armazenadas e decodificadas em *cache*, o que acelera ainda mais sua execução. Outra vantagem é a utilização de paralelismo no nível da instrução, uma vez que são instruções independentes. Os núcleos de processamento ociosos podem ser utilizados para a executar instruções de outras *threads*.

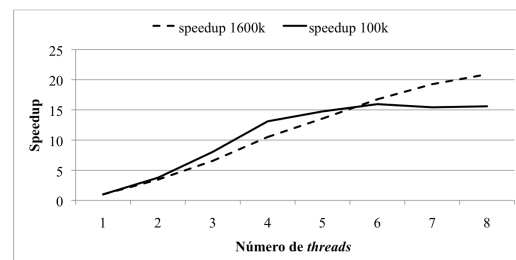


Figura 5. Speedup do JSensor.

No caso do JSensor, ao inicializar duas *threads* quatro núcleos podem ser facilmente utilizados, conferindo aceleração de quatro e não de dois a aplicação. Note que à medida que o número de *threads* JSensor aumenta o nível de paralelismo implícito ou no nível da instrução diminui, pois os núcleos ociosos também diminuem. De qualquer forma, a política de *cache* ainda continua beneficiando o JSensor, portanto para a

entrada com 1.600k sensores a aceleração aumenta até oito *threads*. Já com 100k sensores a aceleração estabiliza com cinco *threads* JSensor. Neste ponto, tanto o benefício da política de *cache* quanto o paralelismo explícito e implícito chegam a um limite. A aceleração JSensor tende a diminuir a medida que a criação de sensores fica mais eficiente, portanto ter aceleração super linear não basta.

Para avaliar a corretude no JSensor, armazenamos o número de mensagens e seus conteúdos de forma a conseguir compará-los ao final de um experimento. Após as simulações, verificamos não somente o número de mensagens, mas também os tempos de envio, recebimento e a ordem de propagação. Observamos que o JSensor e OMNeT++ geraram os mesmos logs. Isso evidencia a corretude das simulações executadas pelo JSensor, uma vez que o OMNeT++ já é um simulador testado e validado.

Alguns pontos importantes que estão em fase de melhoramento é o desenvolvimento de uma versão com memória distribuída, o que nos permitirá executar simulação em diversas máquinas e utilizando diversos núcleos. Além disso, pretendemos incluir diferentes modelos de rede e de aplicação para permitir a maior utilização do JSensor pela comunidade da área de RSSFs. Por fim, com o objetivo de validar os modelos de aplicações em RSSFs pretendemos comparar o JSensor com o TOSSIM, que é um simulador que integra as funcionalidades do TinyOS, um Sistema Operacional proprietário de alguns sensores comerciais.

URL do JSensor e Demonstração planejada para o SBRC

Sítio: <http://www.joubertlima.com.br/JSensor/>.

Iremos demonstrar uma execução completa do JSensor, com definição e explicação dos cenários possíveis. Sendo eles a área de simulação, tamanho do raio de conexão, criação dos nós e escolha dos modelos do usuário. Estabelecidas tais definições, execução da simulação e posterior verificação dos resultados obtidos, tanto os logs como a verificação dos tempos de execução, utilização de memória e *speedup*.

Referências

- Aquino, A. L., Junior, O. S., Frery, A. C., de Albuquerque, E. L., and Mini, R. A. (2013). Musa: Multivariate sampling algorithm for wireless sensor networks. *IEEE Transactions on Computers*, (in press).
- Gustafson, J. L. (1988). Reevaluating amdahl's law. *Communications of the ACM*, 31(5).
- Khan, M., Askwith, B., Bouhafs, F., and Asim, M. (2011). Limitations of simulation tools for large-scale wireless sensor networks. In *IEEE Workshops of International Conference on Advanced Information Networking and Applications (AINA'11)*.
- Lee, H., Manshadi, V., and Cox, D. (2009). High-fidelity and time-driven simulation of large wireless networks with parallel processing. *IEEE Communications Magazine*, 47(3).
- Lewis, P., Lee, N., , Welsh, M., and Culler, D. (2002). Tossim: Accurate and scalable simulation of entire tinyos applications. In *5th Symposium on Operating Systems Design and Implementation*.
- Oliveira, L. M. L. and Rodrigues, J. J. P. C. (2011). Wireless sensor networks: a survey on environmental monitoring. *Journal of Communications*, 6(2).